# FLA 2026 Lecture Notes

Brendan Griffiths

2026-02-13

# Table of contents


**Preface** **4**

**1 An Introduction to Theoretical Computer Science** **5**

**2 Data, Representations, and Problems** **6**
- 2.1 Data . . . 6
  - 2.1.1 Primitive Data . . . 6
  - 2.1.2 Describing Strings . . . 7
  - 2.1.3 Languages . . . 7
- 2.2 Representations . . . 8
  - 2.2.1 Natural Numbers $\mathbb{N}$ in an alphabet of $k$ symbols . . . 9
  - 2.2.2 Integers $\mathbb{Z}$ in an alphabet of $k$ symbols: . . . 9
  - 2.2.3 Rationals $\mathbb{Q}$ in an alphabet of $k$ symbols . . . 10
  - 2.2.4 Booleans . . . 10
  - 2.2.5 Complex Objects . . . 10
- 2.3 Encodings . . . 11
- 2.4 Problems . . . 12
  - 2.4.1 Decision Problem and Computational Problem Equivalence . . . 14
  - 2.4.2 Language and Decision Problem Equivalence . . . 16

**3 Turing Machines I** **17**
- 3.1 A machine that computes . . . 17
- 3.2 Turing Machines . . . 18
  - 3.2.1 Some intuition . . . 18
  - 3.2.2 Formal Definition . . . 20
  - 3.2.3 Computing . . . 21
  - 3.2.4 Computational Problems . . . 24
- 3.3 Turing Machines as Algorithms . . . 26
  - 3.3.1 Analysis of Turing Machines . . . 26
  - 3.3.2 The Church-Turing Thesis . . . 28

**4 Turing Machines II** **29**
- 4.1 Turing Machines and Abstraction . . . 29
  - 4.1.1 Variables . . . 29
  - 4.1.2 Functions / Subroutines . . . 30
  - 4.1.3 Loops . . . 32

4.2 TM Variants . . . 33
4.2.1 Different kinds of Tapes . . . 33
4.2.2 Extended Instruction Sets . . . 36
4.2.3 Multi-tape Turing Machines . . . 38
4.3 Turing Machines as Data . . . 40
4.4 Universality . . . 40

**5 Undecidability by Diagonalisation** **41**
5.1 Countable and Uncountable Infinity . . . 41
5.2 Algorithms and Decision Problems . . . 43
5.2.1 How many algorithms are there? . . . 43
5.2.2 How many decision problems are there? . . . 44
5.2.3 Undecidability . . . 45
5.3 An undecidable problem . . . 46
5.3.1 String Casting . . . 46
5.3.2 Feeding TM strings into TMs . . . 47
5.3.3 Diagonalisation . . . 48

# Preface

This book is a collection of lecture notes for COMS3003A and COMS3021A.

# 1 An Introduction to Theoretical Computer Science

Maybe one day I will write something for this page…

# 2 Data, Representations, and Problems

## 2.1 Data

### 2.1.1 Primitive Data

Symbols: The most basic unit - may be anything that can be Read & Written

- Numbers: $0, 1$
- Latin Letters: $A, B, C, ..., Y, Z$
- Greek Letters: $\alpha, \beta$
- Electricity: $1.5V$, $3.0V$
- Sound

Alphabet: A finite set of symbols

- Common notation $\Sigma$, $\Gamma$, $\Delta$

Strings / Words:

- A finite sequence of symbols
- $\varepsilon$ is the empty word
- Words have a length $|w| = n \iff$ there are $n$ symbols in $w$
  - $|\varepsilon| = 0$

String Operators:

- Concatenation - connecting two symbols or strings
  - $w_1 \circ w_2 = w_1 w_2$
- Repeat Concatenation - concatenate the same string the specified number of times
  - $w^n = w \circ w \circ w \circ ... \circ w$

### 2.1.2 Describing Strings

String Generators: Describe a set of strings

- Union - Either string
  - $w \cup v = \{ w, v \}$
- Kleene Star - Repeat the string any numbers of times
  - $w^* = \{ \varepsilon, w, ww, www, wwww, ... \}$
- Kleene Plus - repeat the string at least once
  - $w^+ = ww^* = \{ w, ww, www, ... \}$
- Otherwise called a regular expression

The set of all strings over an alphabet is

$$\Sigma^*$$

> **Note**
>
> Let $\Sigma = \{ a, b \}$
>
> $$\Sigma^* = \left\{ \begin{array}{c} \varepsilon, \\ a, aa, aaa, ... \\ b, bb, bbb, ... \\ ab, aba, abaa, ... \\ abba, abbaa, abbaaa, ... \end{array} \right\}$$

### 2.1.3 Languages

A language $L$ is a set of strings (i.e. a subset of $\Sigma^*$)

> **Notation**
>
> As shorthand, we may specify a set $L$ is a language by indicating that it is a subset of all strings.
>
> $$L \subseteq \Sigma^*$$

Language Examples:

- The set of all strings with two symbols

$$\{w \in \Sigma^* \mid |w| = 2\}$$

- The set of all strings that end in 0

$$\{w \in \Sigma^* \mid w = v \circ u \text{ and } u = 0\}$$

## 2.2 Representations

A representation is an interpretation of objects into strings

If I give you 5 objects, how would you label them? How about 10 objects? What about an infinite number of objects?

Formally, a representation is a mapping from a set of objects $X$ to a string

$$r : X \mapsto \Sigma^*$$

We also want to add the following properties to our representation:

- Every input has some output

$$\forall x \in X, \exists w \in \Sigma^*, r(x) = w$$

- Every output has at least one input

$$\forall w \in \Sigma^*, \exists x \in X, r(x) = w$$

- If two inputs are different, their outputs must be different

$$\forall x, y \in X, x \neq y \implies r(x) \neq r(y)$$

**A brief interlude on relations**

A relation $\alpha : X \mapsto Y$ is the more general idea of a function. A function $f : X \mapsto Y$ requires that each $x \in X$ maps to only one $y \in Y$. Relations are a relaxation of that to-one property (called functional). Our first property of all $x \in X$ having at least one $y \in Y$ is called serial.
Another way to think of relations is using sets. A binary Relation $R : X \mapsto Y$ is the subset $R \subseteq X \times Y$. Notationally, we say $x \, R \, y$ ($x$ relates to $y$ by $R$) if $(x, y) \in R$
An example is equality of numbers $=: \mathbb{R} \mapsto \mathbb{R}$

$$\forall x, y \in \mathbb{R}, x = y \iff (x, y) \in \{ \ (a, a) \mid a \in \mathbb{R} \}$$

Notation

Instead of writing $r(x)$ we will write $\langle x \rangle_r$ to mark it as a string
If it is clear what representation we are using, then we can drop the $r$

A representation allows us to take random objects and 'insert' them into strings, but we also want to be able to 'extract' an object out of a string.

So we add the additional requirement that our representation must be invertible. We define some default value for $X$. If the string cannot be identified, we just map it to this default value.

Warning

We are hand-waving away a fairly important problem. Two strings may represent the same object, for example in decimal numbers: $0.99999\ldots = 1$
As an exercise, try write a more rigorous definition of a representation using a tool you were taught in abstract maths (hint: equivalence)

### 2.2.1 Natural Numbers $\mathbb{N}$ in an alphabet of $k$ symbols

$$\forall n \in \mathbb{N}, \langle n \rangle_{\mathbb{N},k} = \text{The base-}k\text{ encoding of } n$$

By default, we will use $k = 2$. If the $k$ is left off, then assume it is 2

### 2.2.2 Integers $\mathbb{Z}$ in an alphabet of $k$ symbols:

$$\forall z \in \mathbb{Z}, \langle z \rangle_{\mathbb{Z},k} = \begin{cases} 0 \circ \langle |z| \rangle_{\mathbb{N},k} & \text{if } z \geq 0 \\ 1 \circ \langle |z| \rangle_{\mathbb{N},k} & \text{if } z < 0 \end{cases}$$

Warning

What does the string 00 mean in this representation?
This is where our "default" value for a "garbage" string comes into play. We cannot recogize 00 according to our rules, so we instead just map it to the integer 0.

### 2.2.3 Rationals $\mathbb{Q}$ in an alphabet of $k$ symbols

$$\forall \frac{x}{y} \in \mathbb{Q}, \left\langle \frac{x}{y} \right\rangle_{\mathbb{Q},k} = \langle x \rangle_{\mathbb{Z},k} \# \langle y \rangle_{\mathbb{Z},k}$$

We've had to add the symbol # to separate our integers in the fraction. We can simplify this down into just our alphabet of $k$ symbols.

To pull a rational number out of a string, we use the reverse of the above, but if we cannot recognize the string, we instead default to 0.

Binary simplifier example:

$$e_2(x) = \begin{cases} 00 & \text{if } x = 0 \\ 01 & \text{if } x = 1 \\ 10 & \text{if } x = \# \\ 11 & \text{otherwise} \end{cases}$$

### 2.2.4 Booleans

$$\langle b \rangle_{\mathbb{B},k} = \begin{cases} 1 & \text{if } b = \text{True} \\ 0 & \text{if } b = \text{False} \end{cases}$$

### 2.2.5 Complex Objects

We know have a fairly robust set of primitive representations. We can encode natural numbers, integers, rationals, and booleans, but how can we encode a more complex object? One that is made up of several sub-objects?

If our object has 2 components (imagine a 2D vector), we can employ the same separation as our rationals.

$$\left\langle \begin{pmatrix} x \\ y \end{pmatrix} \right\rangle := \langle x \rangle \# \langle y \rangle$$

For $m$ components?

$$\left\langle \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{pmatrix} \right\rangle := \langle a_1 \rangle \# \langle a_2 \rangle \# ... \# \langle a_m \rangle$$

**What if we have a vector of rationals?**

We use # as a separator for both our vector components and our numerator and denominator. This is a collision of meaning. How can we write something like

$$\begin{pmatrix} \frac{a}{b} \\ \frac{c}{d} \end{pmatrix}$$

Our current encoding of this just becomes

$$\left\langle \begin{pmatrix} \frac{a}{b} \\ \frac{c}{d} \end{pmatrix} \right\rangle = \langle a \rangle \# \langle b \rangle \# \langle c \rangle \# \langle d \rangle$$

which is either a 4D integer vector, a 3D vector in $\mathbb{Z} \times \mathbb{Q} \times \mathbb{Z}$, or a 2D vector of rationals. We can replace the vector separator # with either a new symbol ; or an additional #

$$\left\langle \begin{pmatrix} \frac{a}{b} \\ \frac{c}{d} \end{pmatrix} \right\rangle = \langle a \rangle \# \langle b \rangle \, ; \langle c \rangle \# \langle d \rangle$$

To get back to Binary alphabet we can use the following encoding:

$$e(s) = \begin{cases} 00 \; s = 0 \\ 01 \; s = 1 \\ 10 \; s = \# \\ 11 \; s = ; \end{cases}$$

**What is the smallest alphabet?**

What is the smallest possible alphabet we can construct? How might this alphabet represent different objects?

## 2.3 Encodings

Encodings are the technique we use to write symbols in a different alphabet. We apply them on a semi-regular basis in computer science.

An encoding is a mapping that converts a symbol to a string in another alphabet

$$e : \Sigma \mapsto L \subseteq \Gamma^*$$

$$s \in \Sigma \implies e(s) \in L \subseteq \Gamma^*$$

**Notation**

We can also "encode" strings

$$\text{Let } w = s_1 s_2 ... s_n \in \Sigma^* \text{ and } e : \Sigma \mapsto L \subseteq \Gamma^*$$

$$e(w) = \langle w \rangle_e = e(s_1)e(s_2)...e(s_n) \in \Gamma^*$$

There are two types of encodings:

- Fixed Width: All encoded strings are the same length
  - A new symbol is indicated by reaching the end of the fixed length
  - Example: ASCII

$$e : \Sigma \mapsto \Gamma^n$$

- Variable Width: All encoded strings have different lengths

$$e : \Sigma \mapsto \{w \in \Gamma^* \mid |w| \leq n\}$$

  - A new symbol is indicated in some fashion
    * All symbols start the same way, but feature different symbols for the rest
    * A space between symbols
  - Example: Morse Code
    * Morse Code was originally designed as an auditory alphabet. It has three symbols, a *short sound* (dot), a *long sound* (dash), and *no sound* (seperator). However, we can simplify it into a binary alphabet of sound, no sound by defining a dash as the repetition of sound over a fixed time interval (the standard is 3 times the length of a dot).

**Warning**

All encodings are representations, just from an alphabet of symbols $\Sigma$ to strings $\Gamma^*$ instead of a arbitrary set $X$. Can we make an encoding from a set of strings to another set of strings?

$$e : \Sigma^* \mapsto \Gamma^*$$

## 2.4 Problems

We want to draw a distinction between an algorithm and a problem.

We define a problem, as the class of inputs, and class of outputs for which we are interested in.

$$P : X \mapsto Y$$

An algorithm is the set of instructions by which we manipulate our inputs into our outputs.

A good intuition for the distinction is that algorithms can call other algorithms, but they cannot call a problem.

> **Note**
>
> To rely on some prior programming experience, you can imagine that a problem is a "declaration" in code, while the algorithm is the actual "definition".
> For example, in C++, in some header (.h) file we write
>
> ```
> unsigned int compute_square(unsigned int n);
> ```
>
> which is our "Problem" COMPUTE_SQUARE : $\mathbb{N} \mapsto \mathbb{N}$.
> and then in a .cpp file we have
>
> ```
> unsigned int compute_square(unsigned int n) {
>  unsigned int n_square = n * n;
>  return n_square;
> }
> ```
>
> as our algorithm.
> This is not a perfect one-to-one relationship. Our header definition lacks the description of what our output should look like as the full definition of COMPUTE_SQUARE is
>
> $$\text{COMPUTE_SQUARE}(n) = n^2$$
>
> and implementing it in code is itself an *almost* verbatim description of the problem. Use it as a starting off position.

Examples:

- Prime Factorisation: Input any natural, output: list of prime numbers

$$P : \mathbb{N} \mapsto [\mathbb{N}]$$

- Derivative: Input a differentiable function, output the derivative function

$$P : F \mapsto F'$$

- Swap Two Numbers: Input two naturals, output two naturals swapped

$$P : \mathbb{N} \times \mathbb{N} \mapsto \mathbb{N} \times \mathbb{N}$$

We want to "compute" the answer, so we turn the domain and co-domain into a string with representations $r_X$ and $r_Y$ that we can manipulate

$$P : r_X(X) \mapsto r_Y(Y)$$

A **Computational Problem** is therefore a function $f$ from one set of strings $\Sigma^*$ to another set of strings $\Gamma^*$

$$f : \Sigma^* \mapsto \Gamma^*$$

The most common form however is

$$f : \{\ 0, 1\ \}^* \mapsto \{\ 0, 1\ \}^*$$

Sometimes, we just want to identify if a given object has some property attached to it

- Is this number even?
- Is it a rational?
- Can I take the derivative?

We call these kinds of a problems a **Decision Problem**

$$f : \Sigma^* \mapsto \{\ 0, 1\ \}$$

In fact, we can restructure any computational problem into a decision problem.

### 2.4.1 Decision Problem and Computational Problem Equivalence

As an example: Lets take the computation problem of dividing a natural number

$$\text{Let } n, m \in \mathbb{N}, \text{ and } \frac{n}{m} \in \mathbb{Q}$$
$$\text{COMPUTE_NATURAL_DIV}(n, m) = \frac{n}{m}$$

Let assume an algorithm for COMPUTE_NATURAL_DIV exists, say $A$.

If we give $A$ a string natural number pair $\langle n, m \rangle$ it will output the string $\langle \frac{n}{m} \rangle$

We define the Decision problem with $n, m, q \in \mathbb{N}$

$$\text{IS_NATURAL_DIV}(n, m, q) = \begin{cases} 1 \text{ if } \frac{n}{m} = q \\ 0 \text{ if } \frac{n}{m} \neq q \end{cases}$$

Let $A$ be an algorithm that solves COMPUTE_NATURAL_DIV.

We write a new algorithm $B$ on that solves IS_NATURAL_DIV

define $B$ on input $\langle n, m, q \rangle$

1. If $A(\langle n, m \rangle) = q$
   - True => Return 1
   - False => Return 0

And now for the reverse:

Let $B$ be an algorithm that solves IS_NATURAL_DIV

define $A$ on input $\langle n, m \rangle$:

1. for $q$ in $[1, n]$
   1. if $B(\langle n, m, q \rangle) = 1$
      - True => return q
      - False => do nothing
2. return $\langle 0 \rangle$

Algorithm $A$ solves COMPUTE_NATURAL_DIV as:

1. If $n < m$ then $\frac{n}{m} \notin \mathbb{N}$ and no answer exists or $n = 0$ therefore the loop will never have a $q$ with $B(\langle n, m, q \rangle) = 1$, thus we return $\langle 0 \rangle$
2. If $n = m$ then $\frac{n}{m} = 1$, therefore for $q = 1$, we will return $q$
3. If $n > m$ then $n = qm$
   - The smallest possible $q$ is 1 and $m = n$
   - The largest possible $q$ value is $q = n$ and occurs when $m = 1$
   - If $q > n$ then $m < 1$, thus $m \notin \mathbb{N}$, therefore $B$ does not apply

> **Warning**
>
> Try to do the same for a computational problem $P$. Given $P : \{0,1\}^* \mapsto \{0,1\}^*$ and algorithm $A_P$, show there exists a decision problem $P' : \{0,1\}^* \mapsto \{0,1\}$

### 2.4.2 Language and Decision Problem Equivalence

Languages are equivalent to decision problems. We can construct a decision problem out of language membership, and we can construct language membership out of a decision problem.

$$w \in L \iff D(w) = 1$$

**Note**

Show $D(w) = 1 \implies w \in L$

Let $D : \Sigma^* \mapsto \{0, 1\}$ be a decision problem.

Define the language $L$ as

$$L = \{w \in \Sigma^* \mid D(w) = 1\}$$

Show $w \in L \implies D(w) = 1$

Let $L \subseteq \Sigma^*$ be a language.

Define the decision problem $D$ as

$$D(w) = \begin{cases} 1 & w \in L \\ 0 & w \notin L \end{cases}$$

# 3 Turing Machines I

## 3.1 A machine that computes

Your intuition might be that a computer is a small box that sits on a desk, in a server rack, and even in your pocket. Making use of electronics, digital circuits, and millions of bits and bytes to perform what is sufficiently indistinguishable from magic [1].

In 1936, they had a very different picture. The closest equivalent were mechanical devices that used levers, linkages, and gears to produce work - similar to a train. Instead of work being motion, it would be the evaluation a series of steps, generally finding a solution to some form of arithmetic. Mechanical calculators were probably the most well known instances of a "computer".

https://www.youtube.com/embed/s_hbvRTGcUI

> **Note**
>
> The first general purpose computer was actually proposed about 100 years before Turing in 1837. Charles Babbage designed a machine he called the "Analytical Engine".
> This machine was a digital-mechanical hybrid that made use of components that would not be out of place in a contemporary digital computer. Had it been finished, it would have been the first known instance of device that is Turing Complete [2].
> Ada Lovelace is often called the first programmer because she wrote the first *published* progam for this device. [3] This program, called Note G, calculated a sequence of Bernoulli Numbers.

When we build our mathematical model of a computer and algorithm, I want you to keep in mind the 'mechanical' nature of it.

For our model to be correct, there are important ideas that need to be captured:

1. It should be as simple as possible
    - What is the smallest amount of 'work' we can get away with for each step in our device?

[1] Arthur C. Clarke's third law
[3] citation needed
[3] One day I will find citations for this…

- We will use a string as the only data structure

2. It should be able to do everything an algorithm can do - For loops, if statements, variables etc. should all be implementable on the device

## 3.2 Turing Machines

### 3.2.1 Some intuition

Imagine a physical device that has three components:

1. A space to read and write symbols - a tape of sequential cells
2. A mechanism to perform the read and write - a head that can read or write on a given cell on our tape
3. A control unit to tell the the head what symbol it should be writing - a box that keeps track of our algorithm steps and where along that algorithm where are

- On our tape, we have a set of symbols on every cell - even if that symbol is a "blank" ($\sqcup$) symbol to represent nothing ($\sqcup, 0, 1, 2, ...$).
- We will also assume that our tape has as much space as it needs - i.e. an infinite tape.
- Our head mechanism, can move between the different cells going either left or right along it.
- The control unit is composed of various 'device states' ($q_1, q_2, q_3, ...$) that holds the information about whats happened so far (Gears in different positions, or electronic signals in specific patterns).

This device has only one instruction: A conditional statement based off the current device state $q_i$, and the symbol currently on the tape $s_n$ that tells us what state to go to next $q_j$, what symbol should be written on the tape $s_m$, and whether we should go left or right $D \in \{L, R\}$.

$$q_i, s_n \rightarrow q_j, s_m, D$$

**Note**

If we tried to build this device in the real world, these instructions are just the series of levers and gears that interact to create our desired effect.

- Write this symbol - rotate a ball with all the symbols on it
- Change to the 5th state - rotate this gear by 5 teeth from this notch
- Go left - rotate this other gear counter-clockwise

https://www.youtube.com/embed/yKpIwi1UUIk?si=_2KDfjjzrn8RyoPD&start=115

We will add the stipulation that if our machine enters specific states, the computation will stop: For a Yes / No (Accept / Reject) machine we will have the special states $q_{\text{accept}}, q_{\text{reject}}$ indicating the corresponding answer.

Additionally, we need to start somewhere, so we designate $q_0$ as the 'start state'. If we need to run this device, we must first reset it so that we are in $q_0$.

Our programming language for this device will be all the possible permutations of these instructions that direct the device:

- In $q_0$,
  - reading $\sqcup$, go to $\{q_0, q_1, q_{\text{halt}}\}$, writing symbol $\{\sqcup, 0, 1\}$ and moving $\{L, R\}$.
  - reading 0, go to $\{q_0, q_1, q_{\text{halt}}\}$, writing symbol $\{\sqcup, 0, 1\}$ and moving $\{L, R\}$.
  - reading 1, go to $\{q_0, q_1, q_{\text{halt}}\}$, writing symbol $\{\sqcup, 0, 1\}$ and moving $\{L, R\}$.
- In $q_1$,
  - reading $\sqcup$, go to $\{q_0, q_1, q_{\text{halt}}\}$, writing symbol $\{\sqcup, 0, 1\}$ and moving $\{L, R\}$.
  - reading 0, go to $\{q_0, q_1, q_{\text{halt}}\}$, writing symbol $\{\sqcup, 0, 1\}$ and moving $\{L, R\}$.
  - reading 1, go to $\{q_0, q_1, q_{\text{halt}}\}$, writing symbol $\{\sqcup, 0, 1\}$ and moving $\{L, R\}$.

Our program $\delta$ is the subset of of these permutations that describe the particular behaviour we want.

- In $q_0$
  - reading $\sqcup$, go to $q_1$, writing $\sqcup$ and moving $L$
  - reading 0, go to $q_0$, writing 0 and moving $R$
  - reading 1, go to $q_0$, writing 1 and moving $R$
- In $q_1$
  - reading $\sqcup$, go to $q_{\text{reject}}$, writing $\sqcup$ and moving $R$
  - reading 0, go to $q_{\text{reject}}$, writing 0 and moving $R$
  - reading 1, go to $q_{\text{accept}}$, writing 1 and moving $R$

**Question**

What do these instructions actually do? Try describe the above algorithm in english.
If we use the binary repesentation of integer's, $\langle z \rangle_{\mathbb{Z}}$, what is happening to the number $z \in \mathbb{Z}$?

And a computation is the series of changes that has occured in our device and tape.

1. Set the tape to be the input string 0111,
2. Set the device to be in $q_0$ and the head to be at the first symbol

- We always start at the first symbol of our input

3. Follow each instruction until $q_{\text{accept}}$ or $q_{\text{reject}}$
   - Step 0: In $q_0$, scanning cell 0, reading symbol 0, go to $q_0$, write 0, move right to cell 1
     - $q_0 0111$
   - Step 1: In $q_0$, scanning cell 1, reading symbol 1, go to $q_0$, write 1, move right to cell 2
     - $0q_0 111$
   - Step 2: In $q_0$, scanning cell 2, reading symbol 1, go to $q_0$, write 1, move right to cell 3
     - $01q_0 11$
   - Step 3: In $q_0$, scanning cell 3, reading symbol 1, go to $q_0$, write 1, move right to cell 4
     - $011q_0 1$
   - Step 4: In $q_0$, scanning cell 4, reading symbol $\sqcup$, go to $q_1$, write $\sqcup$, move left to cell 3
     - We have moved off the end of our string into a "blank" cell
     - $0111q_0\sqcup$
   - Step 5: In $q_1$, scanning cell 3, reading symbol 1, go to $q_{\text{accept}}$, write 1, move right to cell 4
     - $011q_1 1$
   - Step 6: In $q_{\text{accept}}$ scanning cell 4 $\rightarrow$ halt
     - $0111q_{\text{accept}}\sqcup$
4. Read the result as 1 as we are in $q_{\text{accept}}$

Congratulations, we have just built a physical device that evaluates some algorithm. If we use $\langle z \rangle_{\mathbb{Z}}$, then this algorithm determines if the given integer is even or odd.

### 3.2.2 Formal Definition

The previous device is essentially an implementation of a Turing Machine as a mechanical machine.

Mathematically, a Turing Machine for Decision Problems are defined as follows

A **Decision Turing Machine** $M$ is the tuple $(Q, \Gamma, \Delta, \delta, q_{\text{init}}, q_{\text{accept}}, q_{\text{reject}})$ where

- $Q$ is a finite non-empty set of *states*
- $\Gamma$ is an **input alphabet** such that $\sqcup \notin \Gamma$
- $\Delta$ is a **tape alphabet**, satisfying the following conditions:

1. $\Gamma \subseteq \Delta$
2. $\sqcup \in \Delta$
3. $Q \cap \Gamma = \emptyset$

- $q_{\text{init}}, q_{\text{accept}}, q_{\text{reject}} \in Q$
- $\delta$ (called the **transition function** or **program**) is a finite set of instructions in the form $q_i, s_n \to q_j, s_m, D$

  1. $q_i, q_j \in Q$
  2. $s_n, s_m \in \Gamma$
  3. $D \in \{L, R\}$
  4. and for all $q \in Q \backslash \{q_{\text{accept}}, q_{\text{reject}}\}$ and all $s \in \Delta$ there exists only one instruction in $\delta$ that has the form $q, s \to q', s', D$

**Question**

We have seen previously that Decision Problems and Computational Problems are equivalent if an algorithm exists for one, then an algorithm must exist for the other. Try to rewrite this proof but using a Turing Machine (See Computation Problems for a definition of Computational Turing Machines) instead of a generic algorithm.

**Notation**

Decision and Computation equivalence means that the exact specifics of our Turing Machine are not that important. We stick to a Decision Turing Machine because it is a stronger continuation from Regular Languages and Context-Free Grammers. However, there is no reason you cannot tackle this entire branch using Computational problems instead.

### 3.2.3 Computing

A **configuration** is the current state of our Turing Machine, where the head is positioned, and what symbols are on the tape.

Let $M$ be a Turing Machine,

- A **configuration** of $M$ and tape is a word $w$ in the alphabet $Q \cup \Delta$ satisfying the following:

  1. $w$ contains exactly one symbol from $Q$
  2. $w$ does not begin with $\sqcup$
  3. If the last symbol of $w$ is a $\sqcup$, the it is preceded by a symbol from $Q$

- A **configuration** is *halting* if it contains $q_{\text{accept}}$, $q_{\text{reject}}$
- A **configuration** is *accepting* if it contains $q_{\text{accept}}$

- A **configuration** is *rejecting* if it contains $q_{\text{reject}}$

Our machine starts in $q_{\text{init}}$ and has the input string written on its tape

Let $M$ be a Turing Machine, and $x \in \Gamma^*$,

The **initial configuration** of $M$ on $x$ is

$$q_{\text{init}}x$$

Computation is the process of moving between different configurations of our machine and tape. In fact, a computation is the sequence of configurations start from the initial configuration until it is in a halting configuration.

Let $M$ be a Turing Machine, and $w$ and $v$ are configurations of $M$,

We say that $w$ yields to $v$ ($w \vdash v$) if one of the following conditions hold:

1. $q, s \rightarrow, q', s', L \in \delta$ and
   - either $w = xtqsy$ and $v = xq'ts'y$ for $t \in \Delta$ and $x, y \in \Delta^*$
   - or $w = qsx$ and $v = q' \sqcup s'x$ for $x \in \Delta*$
2. $q, s \rightarrow, q', s', R \in \delta$ and
   - either $w = xqsy$ and $v = xs'q'y$ for $x, y \in \Delta^*$
   - or $w = xqs$ and $v = xs'q'\sqcup$ for $x \in \Delta*$

> **Note**
>
> Our example in 3.2.1 produces the sequence of configurations
>
> $$q_0 0111 \vdash 0q_1 111 \vdash 01q_1 11 \vdash 011q_1 1 \vdash 0111q_1\sqcup \vdash 0111 \sqcup q_{\text{halt}}\sqcup$$

Let $M$ be a Turing Machine and $x \in \Gamma^*$,

The **computation** of $M$ on $x$ is the sequence of configurations $c_0, c_1, c_2, ...$ such that

1. $c_0$ is the initial configuration of $M$ on $x$
2. $c_i \vdash c_{i+1}$ for all $i \geq 0$
3. The sequence has at most one halting configuration.
4. If the sequence has a halting configuration $c_n$, then $c_n$ is the last configuration in the sequence.

**Notation**

We will use the symbol $C(M, x)$ to refer to the computation of $M$ on $x$.
The length of a computation $|C(M, x)|$ is the number of configurations in a sequence.

We want to identify the **status** of our computation

Let $M$ be a Turing Machine $M$ and $x \in \Gamma^*$

- $M$ **halts** on $x$ if the computation $C(M, x)$ (sequence $c_0, c_1, c_2, ...$) contains a halting configuration
- $M$ **loops** on $x$ if the sequence $c_0, c_1, c_2, ...$ does not contain a halting configuration
- $M$ **accepts** $x$ if the sequence $c_0, c_1, c_2, ...$ contains an accepting configuration
- $M$ **rejects** $x$ if the sequence $c_0, c_1, c_2, ...$ contains a rejecting configuration

**Notation**

$$M(x) = \begin{cases} 1 & \text{if } M \text{ accepts } x \\ 0 & \text{if } M \text{ rejects } x \\ \infty & \text{otherwise} \end{cases}$$

Lastly, we want a way to classify TM's based on whether they accept, reject or loop on all inputs.

Let $M$ be a Turing Machine

- $M$ is a **decider** if it accepts or rejects $x$ for all $x \in \Gamma^*$
- $M$ **decides** / **solves** a decision problem $P : \Gamma^* \mapsto \{0, 1\}$ if $\forall x \in \Gamma^*, M(x) = P(x)$

**Question**

Let $M$ be a Turing Machine

1. If $M$ is a decider, does it solve some decision problem?
2. If $M$ solves a decision problem, is it a decider?

**Notation**

We say that a Turing Machine $M$ **decides** a language $L \subseteq \Gamma^*$ if

$$M(x) = 1 \iff x \in L$$

Two Turing Machines $M$ and $N$ on the same input alphabet $\Gamma$ are equivalent if they have the same input and output behaviour

$$\forall x \in \Gamma^*, M(x) = N(x)$$

Question

How many equivalent Turing Machines are there?
**Proof Sketch**: If I take some Turing Machine $M$, we can add a new state $q$ and make the new machine $M'$. We can make $q$ fall between any state $q_i$ that has some transition to $q_{\text{accept}}$.

1. If $q_i$ has a transition to $q_{\text{accept}}$, make it instead transition to $q$ (leaving the rest of the transition unmodified)
2. $q$ transitions to $q_{\text{accept}}$ on every symbol, leaving the symbol unmodified.

$M'$ has not changed anything about any computation besides add a new configuration $c_{n+1}$ to the sequence. It has not changed the outcome, nor has it changed the tape in anyway. Thus $M'$ is equivalent to $M$.
We can then create $M''$ in the exact same fashion from $M'$. Repeat this ad-infinitum, and we have an infinite number of equivalent Turing Machines.

### 3.2.4 Computational Problems

A Turing Machine for computational problems is very similar to the standard definition. The only difference is that we have a single notable state $q_{\text{halt}}$ instead of $q_{\text{accept}}$ and $q_{\text{accept}}$

A **Computational Turing Machine** $M$ is the tuple $(Q, \Gamma, \Delta, \delta, q_{\text{init}}, q_{\text{halt}})$ where

- $Q$ is a finite non-empty set of *states*
- $\Gamma$ is an **input alphabet** such that $\sqcup \notin \Gamma$
- $\Delta$ is a **tape alphabet**, satisfying the following conditions:
  1. $\Gamma \subseteq \Delta$ - Our input alphabet can be written on the tape
  2. $\sqcup \in \Delta$ - Our tape has a blank symbol
  3. $Q \cap \Gamma = \emptyset$ - The symbols and states are different
- $q_{\text{init}}, q_{\text{halt}} \in Q$
- $\delta$ (called the **transition function** or **program**) is a finite set of instructions in the form $q_i, s_n \to q_j, s_m, D$
  1. $q_i, q_j \in Q$
  2. $s_n, s_m \in \Gamma$
  3. $D \in \{L, R\}$

4. and for all $q \in Q \backslash \{q_{\text{halt}}\}$ and all $s \in \Delta$ there exists only one instruction in $\delta$ that has the form $q, s \to q', s', D$

The definitions of computation and status of a Computational TM remain the same as a Decision TM. However, we will use a slightly different configuration to define these objects:

- A **configuration** is halting if contains $q_{\text{halt}}$

Computational Turing Machines need to provide some form of output string that we can read off the tape. To be consistent, we say that our resultant string is the string from the tapes head until the next blank symbol that is composed of only input symbols $\Gamma$.

Let $M$ be a Computational Turing Machine.

We define the **result** of a computation of $M$ on $x$ as the string $w$ satisfying the following properties

- $w \in \Gamma^*$
- $w$ is written on the tap
- $w$ is immediately followed by $\sqcup$
- $M$ is scanning the first symbol of $w$

Notation

$$M(x) = \begin{cases} w & \text{if } C(M, x) \text{ halts} \\ \infty & \text{otherwise} \end{cases}$$

Computational TM's can also **solve** some Computational Problem $P : \Gamma^* \mapsto \Gamma^*$. However we use the term $M$ **computes** $P$ to specify it is a computational problem.

Let $M$ be a Computational Turing Machine and $P : \Gamma^* \mapsto \Gamma^*$ a computational problem

- $M$ **computes** $P$ if
$$\forall x \in \Gamma^*, M(x) = P(x)$$

and this gives us a fairly strong definition for when something is computable or not.

Let $f : \Sigma^* \mapsto \Sigma^*$ be a function.

- $f$ is **computable** if there exists some Turing Machine that computes $f$

## 3.3 Turing Machines as Algorithms

### 3.3.1 Analysis of Turing Machines

When we perform an analysis of an algorithm, we typically focus on two resources available to our algorithm:

- Time Complexity - How long will it take to run our algorithm
- Space Complexity - How much memory do we need for our algorithm

If we want to use Turing Machines as a mathematical model for our algorithms, we need to define these same tools.

#### 3.3.1.1 Time

Our Turing Machine has only the single instruction. If we build this machine in reality, then it will take a roughly fixed amount of time to execute this instruction, no matter how many times we do it.

- If we built this device using mechanical components, this time is how long it takes to rotate the gears, manipulate the linkages and so on.
- If we built it use electronic signals, it would be the time to change the voltage of our circuit.

This means we can discard the actual unit and instead count the number of times an instruction is executed. We can recover the actual execution time by taking an average time to execute a single instruction and multiply it by the number of instructions.

A single instruction allows us to move between the different configurations of our machine, thus the number of instructions is the number of configurations in a computation.

Let $M$ be a Turing Machine and $x \in \Gamma^*$,

The **computation time** of $M$ on $x$ is the length of the computation.

$$T(M, x) = |C(M, x)|$$

$T(M, x)$ allows us to analyse the execution of a single input, but we want to understand how it runs on all inputs. However, if two strings are of equal length, then the machine will do the roughly the same *amount* of work, even if the result of that work is different. Instead of the individual strings we analyse the length of the strings.

Let $M$ be a Turing Machine that will halt on every input,

The **time function** $t : \mathbb{N} \mapsto \mathbb{N}$ of $M$ is

$$t(n) = \max\{T(M, x) = |C(M, x)| : x \in \Gamma^n\}$$

In english, our time function for a given input $n$ is the maximum computation time across all strings of length $n$.

> **Note**
>
> We use the maximum time because some inputs of the same length may occasionally do less work.
> Consider the following python script
>
> ```
> x = input()
> for i in range(0, len(x))
>  if x[i] == "0":
>  break
> ```
>
> If we input "111", the for loop will be executed 3 times. If we instead input "101" it will only be executed twice, even though the strings are the same length.
> When analysing this algorithm, we want to know the worst it will do at a given length, not the best. So we take $t(3) = \max\{1, 2, 3, 3\} = 3$ (Inputs: $\{0xx, 10x, 110, 111\}$ where $x \in \Gamma$)

#### 3.3.1.2 Space

In a digital computer, space is measured in the number of bits used. We can do the same here, but instead of bits we will use our tape instead of transistors.

Let $M$ be a Turing Machine and $x \in \Gamma^*$,

The **computation space** $S(M, x)$ of $M$ on $x$ is the number of unique tape cells accessed by our machine.

This is more difficult to tie down an exact value, but we can define some bounds.

The machine can only move left or right on an instruction, it cannot stay in place. Even if we use the empty string, the machine scans a cell in the initial configuration, and scans the next cell after the instruction that may let it halt.

$$q_{\text{init}}, \sqcup \to q_{\text{halt / accept / reject}}, s, D$$

For any other string, we can do the same, or just move back and forth between these two cells infinitely. So the smallest number of unique cells we can use is 2. And if we never re-use a cell

i.e. every instruction accesses a new cell, then we can only access as many cells as there are instructions.

$$2 \leq S(M, x) \leq T(M, x)$$

For the same reasons as time, we want to study the lengths of a string and how the machine behaves, so we define our **space function** in a similar manner.

Let $M$ be a Turing Machine that will halt on every input,

The **space function** $s : \mathbb{N} \mapsto \mathbb{N}$ of $M$ is

$$s(n) = \max\{S(M, x) : x \in \Gamma^n\}$$

### 3.3.2 The Church-Turing Thesis

At this point, you should have an understanding that Turing Machines are a fairly strong model for algorithms. We can analyse them the same way as algorithms, we can program them using instructions, we can build them in the real world (this is a surprisingly important idea), and there is more yet to come.

However, we have not really discussed if a Turing Machine is an exact model of an algorithm. To jump the gun, yes. Turing Machines are an incredibly robust model, and every definition of computer that has been developed in the last 100 years has been shown to be equivalent to at least some form of Turing Machine. Even digital computers with Random-Access Memory are just slightly faster versions of a Turing Machine. In Turing Equivalency, we will study this idea more concretely.

To formulate this, we have developed a simple thesis.

> Every computational problem solvable by an algorithm, can also be solved by a Turing Machine.

This is the Church-Turing Thesis. It allows us to assert that an algorithm *is* a Turing Machine. If we can't write a Turing Machine, then we can't write an algorithm and vice versa.

This is not a mathematical statement however. It converts an intuitive concept (an algorithm - sequence of steps) and converts it into a mathematical object (a TM - sequence of instructions). We cannot really prove this assertion, we only take it as true so long as it appears to be true.

# 4 Turing Machines II

## 4.1 Turing Machines and Abstraction

Writing a Turing Machine can be a very complicated procedure. While it is technically possible to write a Turing Machine to solve something such as IS_PRIME($n$), the actual machine may be very large, and difficult to write manually. To solve this problem, we rely on abstract descriptions of our Turing Machines with the understanding that if it was absolutely necessary we could turn it into the actual TM.

To help us with this abstraction, we can build up a few common primitives used in most algorithm descriptions and show how they may be converted into a Turing Machine.

### 4.1.1 Variables

A variable $x$ is an area of memory that can hold values. In a computer, these variables are of a fixed size - 32 or 64 bits. For a Turing Machine, we will state that variables are of an initial size, but can grow in size as they need to. This is similar to how a C++ std::vector handles it's memory, but instead of having elements of an array, we have a list of cells. If we run out of space, we can extend the variable by moving any other work on the tape.

A variable $x$ in a Turing Machine is the string $\langle x \rangle$ on our tape between the symbol's $x$ and $\square$. We use $x$ as a symbol on our tape to indicate that this region of memory is for the variable $x$. You can come up with some other variable labelling scheme to make it more flexible, but this works if our machine only needs a fixed number of variables.

To modify our variable we need to perform the following:

1. Move to the variable on the tape
2. Update the variable as needed
3. If the new value requires more space then allocated
    - Move *every* symbol to the right of variable over by twice the current length of $\langle x \rangle$
    - Move the end of variable marker to the new position
    - Update the variable as needed.

### 4.1.2 Functions / Subroutines

In our variable allocation, we required a series of subroutines to update the value:

- Move all the symbols over by $k$ cells
- Updating the variable - adding $z$ to an integer, writing a new value in the cells etc.

How can we actually perform this on our Turing Machine?

First, we can write these 'subroutines' as other Turing Machines that just perform the task we need. In the below figures, these machines are drawn abstractly: the squiggly arrows refer to some intermediatry states and transitions that occur between the initial and halting states.

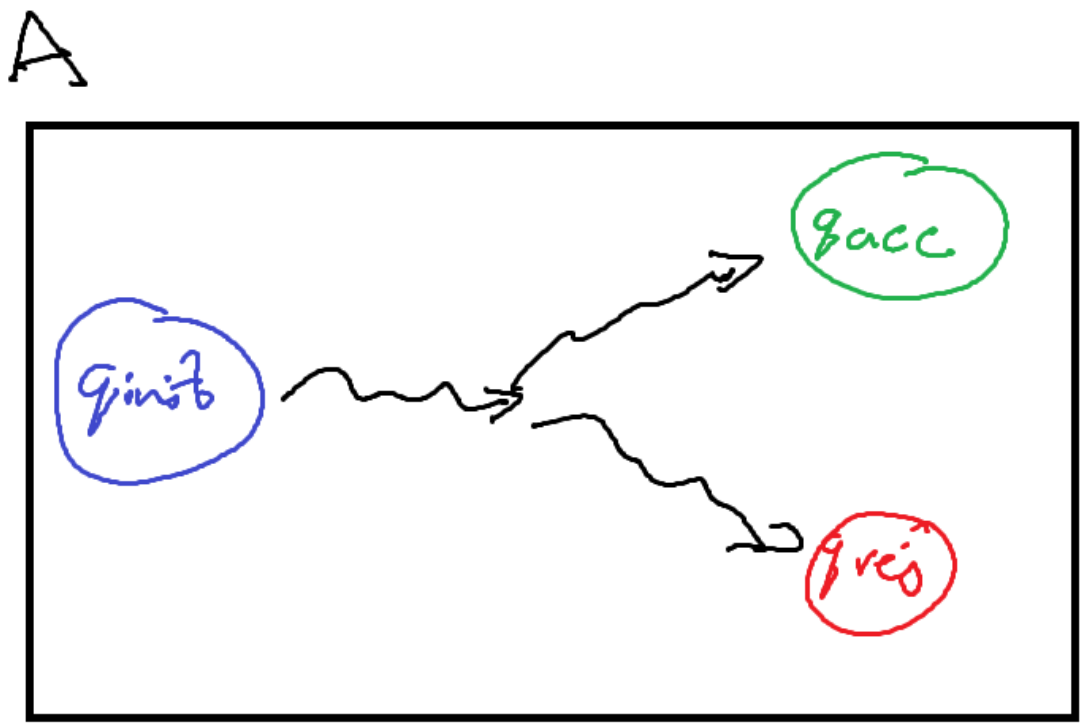


Figure 4.1: An abstract Decision Turing Machine called $A$

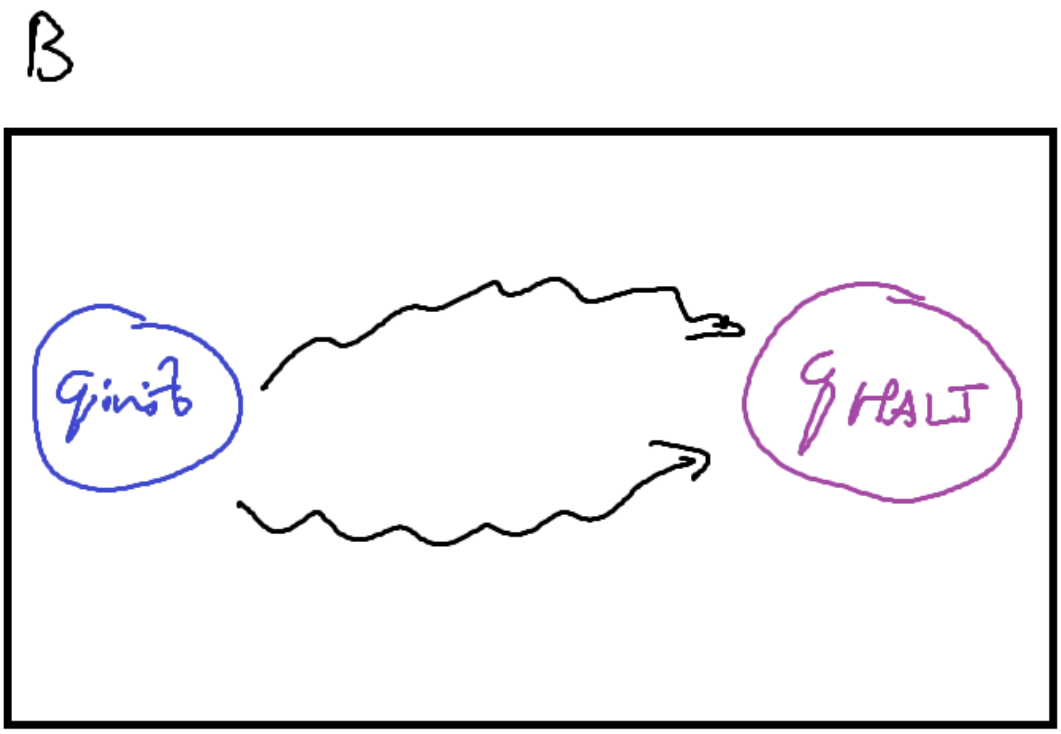


Figure 4.2: An abstract Computational Turing Machine called $B$

To call these machines as a subroutine, our primary machine $M$ must provide some transition into the initial state of the subroutine machine $A$ / $B$, and then provide transitions out from $A$'s ($B$'s) halting state. The machines may need some additional logic to connect them up such as moving into the correct position or cleaning up any left over computation on the tape, but these are relatively simple 'bridging' programs.

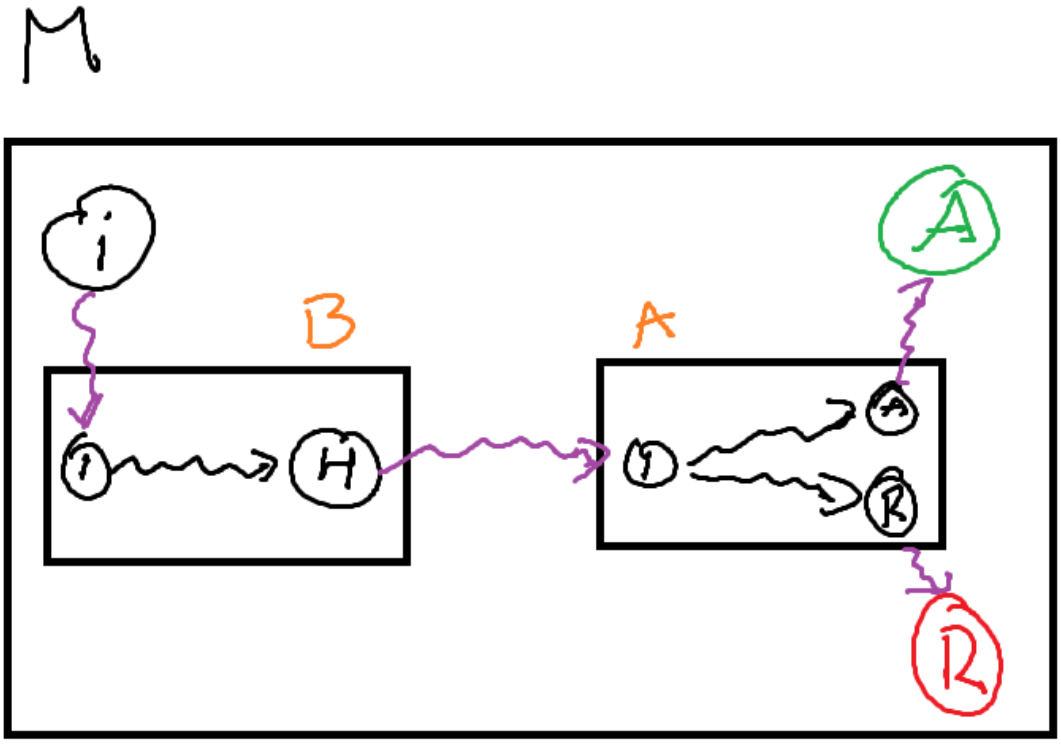


Figure 4.3: An abstract Decision Turing Machine $M$ that uses $A$ and $B$ as subroutines

We can use these subroutines to build large and complex Turing Machines out of a relatively small 'standard library' of other machines.

### 4.1.3 Loops

We can implement a loop in much the same way. The most common form of a `while` loop requires two components:

1. A condition to test for completion
2. A body to execute if it is the condition is true

If we treat each of these components as a subroutine, we can implement a `while` loop. We can use a Decision Turing Machine as our condition

- If it accepts, our condition is true and we execute the body of the loop
- If it rejects, our condition is false and we continue into the rest of the machine

The body is some other computational turing machine, that when it is finished (the subroutine machine halts), we start testing the condition of the loop.

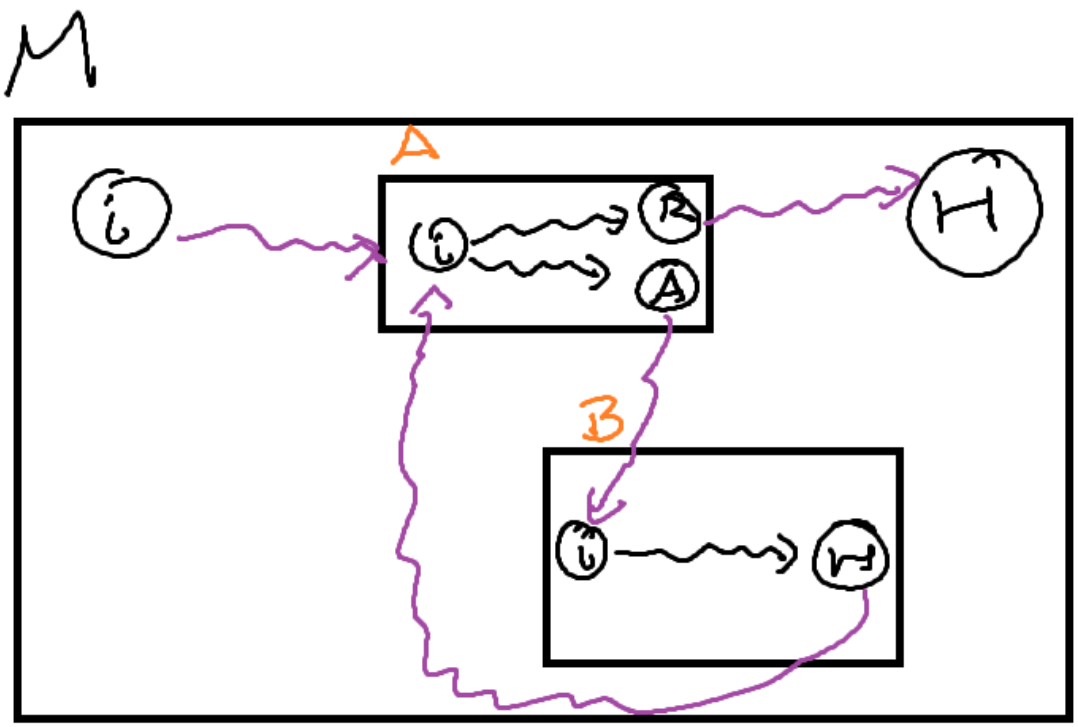


Figure 4.4: An abstract Computational Turing Machine $M$ that uses $A$ and $B$ as a while loop

## 4.2 TM Variants

### 4.2.1 Different kinds of Tapes

When we defined a Turing Machine, we described the tape as infinite in length. The reason for this was to clarify that any algorithm would have as much space as it possibly needs. We never have to worry about running out of memory.

The standard description of a Turing Machine is that we can infinitely in either direction. This is akin to the integers on a number line.

$$\mathbb{Z} = (-\infty, \infty)$$

However, we know that there are other kinds of infinite such as the naturals.

$$\mathbb{N} = [0, \infty)$$

So it stands that we can construct different kinds of infinite tapes for a Turing Machine. The first kind is what we have used before and the normal definition: A tape that is infinite in two directions - bidirectional. The second kind is a tape that is infinite in only one directin - unidirectional.

A Turing Machine with a bidirectional tape is called a **two-way infinite tape** Turing Machine, or a Two-Way TM

<insert diagram of two way tape turing machine>

A Turing Machine with a unidirectional tape is called a **one-way infinite tape** Turing Machine, or a One-Way TM.

<insert diagram of one way tape turing machine>

> **Tip**
>
> Be careful when discussing a unidirectional tape. It does *not* mean that our tape head can only move in a single direction, but that there is some leftmost cell that the tapehead cannot move past.

The One-Way TM works by adding a new symbol, $\triangleright$, to its tape alphabet to mark that we are at the end of the tape. This new symbol $\triangleright$ is placed on the leftmost cell of our tape. We then add the following instruction to *every* state:

$$\forall q \in Q \; q, \triangleright \rightarrow q, \triangleright, R$$

For clarity, when we write an input string $x$ on our tape, we place it after the end-of-tape marker. This means that the initial configuration of a one-way tape Turing Machine is

$$c_0 = \triangleright q_{\text{init}} x$$

What this means is that our TM will 'bounce' off the end-of-tape marker whenever it reaches the leftmost cell. It will never be able to move beyond that cell.

> **The direction of a One-Way TM**
>
> Something to note is that english readers might place the end-of-tape marker $\triangleright$ on the left most cell as we read from left to right. There is no explicit reason for this beyond preference, so it is entirely possible to make use of a different symbol $\triangleleft$ that is on the rightmost cell instead.

$$\forall q \in Q \; q, \lhd \rightarrow q, \lhd, L$$

This would mean that the Turing Machine has a unidirectional tape to the left instead of the right.
Later we will see how this convention can be used.

**Bounded Tapes**

A special kind of Turing Machine called a **Linear Bounded Turing Machine** has a finite tape. The exact definition is similar to a One-Way TM, but with both a left and right end-of-tape marker.

We have constructed two different types of Turing Machine. Can they do the same things?

**Question**

What does it mean for two Turing Machines to do the same thing? Previously we have seen a definition for equivalent Turing Machines:

$$\forall x \in \Gamma^*, M(x) = N(x)$$

Is this definition sufficient to show that different kinds of Turing Machines are equivalent?

To show that these two types of Turing Machine are equivalent, we can describe the process by which we can convert a Two-Way TM into a One-Way TM and vice versa.

**Show that any One-Way TM can be converted into a Two-Way TM**

**Proof sketch**: *We want to recreate the behaviour of a two-way machine on a one-way machine. To accomplish this, if our tape head on the two-way machine needs to move left, and no more space is available, we need to provide that needed space. If we shift every accessed cell over by one, we have 'appended' a new blank symbol to the right, and our machine can continue moving left.*

Let $M$ be a two-way TM, define a one-way TM $M'$ as follows:

1. If the head is not at the leftmost cell, execute the transition as usual.
2. If the transition would move the head to the leftmost cell (Scanning $\rhd$), then
    1. Move every cell used so far on the tape, one cell to the right
    2. Set the leftmost cell before end-of-tape to blank

**Show that any Two-Way TM can be converted into a One-Way TM**

**Proof sketch**: *A one-way TM starts in the initial configuration of $\rhd q_{init} x$. So when our two-way TM starts, we can recreate the one-way behaviour by manually adding an end-of-tape*

*marker to the left of the input tape. Then, for all states we can add the instruction to bounce off the end-of-tape marker. Essentially, we are saying that even though there is infinite memory in the left direction, we will not make use of it.*

Let $M$ be a one-way TM, define a two-way TM $M'$ as follows:

Let $Q' = Q \cup \{q'_{\text{init}}\}$ where $q'_{\text{init}}$ is the initial state of $M'$. Add the following instruction to every state $q \in Q$ execpt $q'_{\text{init}}$:

$$q, \triangleright \to q, \triangleright, R$$

Define the following instruction for $q'_{\text{init}}$:

$$\forall s \in \Delta,\ q'_{\text{init}}, s \to q'_{\text{init}}, s, L$$
$$q'_{\text{init}}, \sqcup \to q_{\text{init}}, \triangleright, R$$

Question: Comparing running times

For both of the above proofs, what is the running time of $M'$ compared to $M$?
**The time function of the two-way TM $M'$ compared to the one-way TM $M$**
Let $t(n)$ be the time function of the one-way TM $M$ Our new machine $M'$ leaves the machine mostly the same but right at the beginning it does two things: 1. Immediatly moves left one cell 2. Writes $\triangleright$ and moves right
and then continues the computation as $M$ would.
So $M'$ executes only two more instructions then $M$, so

$$t'(n) = t(n) + 2 = O(t(n))$$

**The time function of the one-way TM $M'$ compared to the two-way TM $M$**
Let $t(n)$ be the time function of the two-way TM $M$
The new machine $M'$ leaves $M$ mostly unchanged except that any time it moves left onto the end-of-tape marker, it shifts every cell over once to the right.
The worst case scenario is when $M$ always moves left, so for every step $M'$ takes, it must execute the subroutine of shifting the cells over. We need to shift the cells over $t(n)$ times. This shift operation is linear so the runtime of a single shift is the total number of cells accessed so far, or the space function $s(n)$. $s(n)$ is bounded above by $t(n)$ so in the worst case, the shift operation is $O(t(n))$
Therefore,

$$t'(n) = t(n) \cdot O(t(n)) = O(t^2(n))$$

### 4.2.2 Extended Instruction Sets

An important part of the original pitch for a Turing Machine as a mathematical model of computation was that it was simple. One of the componets of this simplicity is in the definition

of an instruction.

$$q, s, \rightarrow q', s', \{L, R\}$$

Our tape head can only ever move left or right a single cell. The instruction does not provide a way to stay in place, nor to skip multiple cells at a time.

However, if we ignore the simplicity requirement, we can add additional movement to the tape head.

A Turing Machine with a stationary movement is called a **stationary** Turing Machine, and has instructions of the form

$$q, s, \rightarrow q', s', \{L, S, R\}$$

What this looks like as configuration with the instruction $q_i, v_1 \rightarrow q_j, v_1', S$,

$$w\, q_i\, v_1 v_2 ... v_n \vdash w\, q_j\, v_1' v_2 ... v_n$$

Does this stationary movement make it solve any additional problems that the standard Turing Machine cannot? No, as we can recreate that stationary movement in a standard Turing Machine by moving left and then right over two configurations.

$$q_i, s, \rightarrow q_j, s', S \text{ becomes} \tag{4.1}$$

$$q_i, s \rightarrow q_i', s', R \text{ and } \forall \sigma \in \Delta, q_i', \sigma \rightarrow q_j, \sigma, L \tag{4.2}$$

$$w\, q_i\, v_1 v_2 ... v_n \vdash w v_1'\, q_i'\, v_2 ... v_n \vdash w\, q_j\, v_1' v_2 ... v_n$$

<diagram of stationary instruction being modified into standard instruction>

We can do the exact same thing for a movement that steps twice in one instruction.

A Turing Machine with a two-step movement is called a **two-step** Turing Machine, and has instructions of the form

$$q, s, \rightarrow q', s', \{LL, L, R, RR\}$$

As a configurations following the instruction $q_i, v_1 \rightarrow q_j, v_1', RR$,

$$w\, q_i\, v_1 v_2 v_3 ... v_n \vdash w v_1' v_2\, q_j\, v_3 ... v_n$$

We can use the same idea from stationary machines for simulating two step movement on a standard Turing Machine

$$q_i, s, \rightarrow q_j, s', RR \text{ becomes} \tag{4.3}$$
$$q_i, s \rightarrow q_i', s', R \text{ and } \forall \sigma \in \Delta, q_i', \sigma \rightarrow q_j, \sigma, L \tag{4.4}$$
$$\tag{4.5}$$
$$q_i, s, \rightarrow q_j, s', LL \text{ becomes} \tag{4.6}$$
$$q_i, s \rightarrow q_i', s', L \text{ and } \forall \sigma \in \Delta, q_i', \sigma \rightarrow q_j, \sigma, L \tag{4.7}$$

The instruction $q_i, v_1 \rightarrow q_j, v_1', RR$, is now executed as

$$w\, q_i\, v_1 v_2 v_3 ... v_n \vdash w v_1'\, q_i'\, v_2 v_3 ... v_n \vdash w v_1' v_2\, q_j\, v_3 ... v_n$$

<diagram of two step instruction being modified into standard instruction>

There is no particular reason to limit ourselves to just two steps. We can define a **$k$-step** Turing Machine, which has instructions of the form

$$q, s, \rightarrow q', s', \{L^k \mid k > 0\} \cup \{R^k \mid k > 0\}$$

A **multi-step** Turing Machine is the umbrella term for a $k$-step Turing Machine with $k > 1$.

Proof practice

Try prove that a $k$-step Turing Machine is equivalent to a standard Turing Machine by induction.
We have seen the base case that $k = 2$ is can be simulated by a standard ($k = 1$) Turing Machine. So now all you need to prove is that a $k + 1$-step TM can be simulated by a $k$-step TM.

Question

We've seen above that simulating the stationary and two-step instructions require additional instructions, and thus configurations, in a standard TM.
Identify the time functions of both the multi-step and stationary machines, as well as the standard machines that recreate the behaviour. What advantages are provided by the additional instructions?

### 4.2.3 Multi-tape Turing Machines

All of our previous machines have only ever had a single tape. Truthfully, this is because of the same pursuit for simplicity as our simple instructions.

< diagram of 2-tape turing machine >

A $k$-tape Turing Machine has $k$ infinte tapes, where each tape has its own head to read and write symbols as well as move.

Our instructions for $k$-tape Turing Machines change to be a conditional statement based off the symbols on *every* tape currently being scanned. Additionally they need to specify what we write on *every* tape, and the direction to move the head on *every* tape.

$$q, \begin{pmatrix} s_1 \\ s_2 \\ \dots \\ s_k \end{pmatrix} \to q', \begin{pmatrix} s'_1 \\ s'_2 \\ \dots \\ s'_k \end{pmatrix}, \begin{pmatrix} D_1 \\ D_2 \\ \dots \\ D_k \end{pmatrix} = q, \vec{s} \to q', \vec{s'}, \vec{D}$$

Multi-tape Turing Machines are convenient as they allow for information to be shared across tapes, and make it faster to access that information. However, they are significantly more complex to write.

In a standard Turing Machine, there needs to be $|\delta| = (|Q| - 2) \times |\Delta|$ instructions. A $k$-tape Turing Machine has $|\delta| = (|Q| - 2) \times |\Delta|^k$ instructions. We need an instruction for every combination of symbols being read across all the tapes.

Do these multi-tape Turing Machines provide any additional capability over a standard Turing Machine? Again, no. We can simulate a $k$-tape Turing Machine on a standard TM by segmenting the single tape into $k$ segments. We can distinguish between tapes by end-of-tape markers $\triangleright$ and $\triangleleft$, and now we can rely on the same logic as our One-Way and Two-Way TM's to simulate the infinite space of these tapes.

< diagram of two tapes on a single tape >

The last thing we need to consider is how do we update the tape head positions and symbols on the different tapes. For every one instruction in the $k$-tape TM, we need to move to each tape segment and update it accordingly.

On each tape, we can record the head position by a special symbol: If the head is currently scanning a cell with symbol $s$, we write $\dot{s}$.

To process an instruction,

1. Move to the first tape
2. For each tape
    1. We scan for the $\dot{s}$ symbol on the current tape
    2. Update it to the new symbol
    3. Remove the dot
    4. Move to the next head position,
    5. Update the new head position symbol $s$ to $\dot{s}$

6. Move on to the next tape

## 4.3 Turing Machines as Data

## 4.4 Universality

# 5 Undecidability by Diagonalisation

## 5.1 Countable and Uncountable Infinity

Up until this point, we have touched on the distinction between the countable infinity $\aleph_0$, and uncountable infinity $\mathfrak{c}$. However, to understand the distinction between Decision Problems and Algorithms concretely, we need to understand *how* different they are.

To begin with, we need the two definitions of countable and uncountable sets:

An infinite set $A$ is **countable** if there exists a bijective function $\alpha$ between $A$ and the natural numbers $\mathbb{N}$, and an infinite set $B$ is **uncountable** if there is no bijection between $B$ and $\mathbb{N}$.

Question - Why does it need to be a bijective function?

The surjective property

$$\forall n \in \mathbb{N}, \exists x \in X \text{ such that } \alpha(x) = n$$

allows us to state that *every* natural number has at least one element in $X$.
The injective property

$$\forall x \in X, \exists! n \in \mathbb{N} \text{ such that } \alpha(x) = n$$

means that some $x$ only ever has one natural number.
For every object, there is only one natural number, and every natural number has some object. Thus every natural number must have only one object, and there is as many objects as there are natural numbers.

Intuitively, we can recognize that there are more real numbers then there are natural numbers. It is tempting to believe that this is because of density (between any two real numbers, we can find some other real number) but this property also applies to the rational numbers.

Prove that the set of rational numbers is dense

As a start, for any two rational numbers $\frac{a}{b} < \frac{c}{d} \in \mathbb{Q}$, show that we can construct some

rational number $\frac{x}{y} \in \mathbb{Q}$ such that

$$\frac{a}{b} < \frac{x}{y} < \frac{c}{d}$$

$$\frac{x}{y} = ???$$

We need to find some other property of countable sets that we can use to demonstrate the size difference. This property is enumeration.

We can **enumerate** the set of natural numbers in a fairly simple manner: The next natural number is the previous number plus one

$$n = m + 1$$

This enumeration means we can theoretically list out all natural numbers.

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...$$

We can also enumerate *any* countable set $A$ using this idea, and our bijection $\alpha : A \mapsto \mathbb{N}$.

$$\begin{aligned} &0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ... \\ &\alpha(0), \alpha(1), \alpha(2), \alpha(3), \alpha(4), \alpha(5), \alpha(6), \alpha(7), \alpha(8), \alpha(9), ... \end{aligned}$$

If we can show that there is no way to enumerate the set of all real numbers, then the bijection that constructs the enumeration must also not exist.

---

**Theorem:** The set of all real numbers has larger cardinality then the set of natural numbers.

Assume, for contradiction, that the subset $[0, 1] \subset \mathbb{R}$ is countable. Then we can enumerate these numbers.

$$x_1, x_2, x_3, x_4, x_5, ... \text{ where } x_i = \alpha(i), i \in \mathbb{N}$$

We can represent a real number $x \in [0, 1]$ as the *infinite* binary sequence by representing each digit as a binary word. One such representation is to tie the value of each digit to where along the interval $x$ is found (More info here).

Using our binary enumeration, we can construct the following table

$$
\begin{array}{l|l}
x_1 & 0\,0\,0\,0\,0\,0\ldots \\
x_2 & 1\,1\,1\,1\,1\,1\ldots \\
x_3 & 1\,0\,1\,0\,1\,0\ldots \\
x_4 & 0\,1\,0\,1\,0\,1\ldots \\
x_5 & 0\,1\,0\,0\,0\,1\ldots \\
\vdots &
\end{array}
$$

Now for every row in this table, $x_i$, we move to the $i$-th digit (along the **diagonal**) and take the opposite that value and make a new binary string $x'$

$$
\begin{array}{l|l}
x_1 & \color{red}{0}\,0\,0\,0\,0\ldots \\
x_2 & 1\,\color{red}{1}\,1\,1\,1\ldots \\
x_3 & 1\,0\,\color{red}{1}\,0\,1\ldots \\
x_4 & 0\,1\,0\,\color{red}{1}\,0\ldots \\
x_5 & 0\,1\,0\,0\,\color{red}{0}\ldots \\
\vdots & \\
\hline
x' & \color{blue}{1\,0\,0\,0\,1}\ldots
\end{array}
$$

The enumeration would eventually list all the real numbers $x_i \in [0, 1]$. For any $x_i$, we can find some digit in $x'$ where $x'$ is different to $x_i$. Thus $x'$ is different from every $x_i$ and as it is *not* apart of the enumerated list, $x' \notin [0, 1]$. However, $x'$ *is* an infinite binary string so it represents some real number in $[0, 1]$.

1. $x' \notin [0, 1]$
2. $x' \in [0, 1]$

This is a contradiction, and thus our original assumption must be false. Therefore $[0, 1]$ is not a countable set. If $[0, 1] \subset \mathbb{R}$ is not countable, then $\mathbb{R}$ must also not be countable.

---

## 5.2 Algorithms and Decision Problems

### 5.2.1 How many algorithms are there?

We know that we can represent a Turing Machine $M$ as the binary string $\langle M \rangle$. So the **set of all *non-equivalent* Turing Machines** $\mathbb{M}$ can be converted into a set of strings $\langle \mathbb{M} \rangle \subseteq \{0, 1\}^*$ As an upperbound, there can only be as many Turing Machines as there are strings.

$$|\mathbb{M}| \leq |\{0, 1\}^*|$$

We also know that there are an infinite number of Turing Machines.

$$|\mathbb{M}| \geq \aleph_0$$

> ⚠ Why can we make this claim?
>
> Take a look at the proof sketch for equivalent Turing Machines in Lecture 3. Can we use a similar idea to show that there are at least an infinite number of *non-equivalent* Turing Machine?

To find the size of $\mathbb{M}$, we need to find how many strings there are. Fortunately, we know this already.

The bijection between $\{0,1\}^*$ is just the representation for binary natural numbers $\langle n \rangle_{\mathbb{N},2}$. Therefore, there are as many binary strings as there are natural numbers.

$$|\{0,1\}^*| = |\mathbb{N}| = \aleph_0$$

All together now,

$$\aleph_0 \leq |\mathbb{M}| \leq \aleph_0 \implies |\mathbb{M}| = \aleph_0$$

Therefore, there are a countably infinite number of Turing Machines.

### 5.2.2 How many decision problems are there?

Every decision problem $P : \{0,1\}^* \mapsto \{0,1\}$ has an equivalent language $L \subseteq \{0,1\}^*$. We need to identify how many languages there are.

A language $L$ is by definition a subset of the set of all strings $\{0,1\}^*$, so it is also a member of the powerset of all strings $\mathcal{P}(\{0,1\}^*)$ (The set of all subsets).

$$L \subseteq \{0,1\}^* \implies L \in \mathcal{P}(\{0,1\}^*)$$

The cardinality of a powerset of some set $A$ is $2^{|A|}$. So the cardinality of $\mathcal{P}(\{0,1\}^*)$ is $2^{\aleph_0}$.

The set of decision problems is larger then the set of natural numbers.

> ℹ Cardinality of the powerset of some finite set example
>
> Let $A = \{0,1,2\}$

$$\mathcal{P}(A) = \{\emptyset, \{0\}, \{1\}, \{2\}, \{0,1\}, \{0,2\}, \{1,2\}, \{0,1,2\}\}$$
$$|A| = 3$$
$$|\mathcal{P}(A)| = 8 = 2^3 = 2^{|A|}$$

### 5.2.3 Undecidability

The last two sections have identified the following facts:

1. $|\mathbb{M}| = \aleph_0$
2. $|\mathcal{P}(\{0,1\}^*)| = 2^{\aleph_0}$

Putting these two statements together, we get,

$$|\mathbb{M}| < |\mathcal{P}(\{0,1\}^*)|$$

There are more decision problems then there are non-equivalent Turing Machines. This doesn't give us enough information to conclude that some problems don't have a deciding Turing Machine. It is entirely possible that one Turing Machine is capable of solving multiple different decision problems.

So we need to show that not every decision problem has some decider. We can do this with another proof by contradiction.

---

**Theorem** There are decision problems that are not decidable.

Assume, for contradiction, that every decision problem is decidable. Therefore, for every $P : \{0,1\}^* \mapsto \{0,1\}$, there is some $M_P \in \mathbb{M}$ that decides $P$.

Let $f$ be a relation that links a decision problem $P$ to its decider $M_P$.

- We assumed that every decision problem is decidable, so $f$ is total - every input has at least one output.
- The definition of $\mathbb{M}$ was that it was the set of all non-equivalent Turing Machines, so $f$ is also injective - every output has only one input.

If there is an injective, total relation $\alpha$ between two sets $X$ and $Y$, then $|X| \leq |Y|$. Therefore, by $f$,

$$\mathcal{P}(\{0,1\}^*) \leq \mathbb{M}$$

> **Note**
>
> We don't have surjectivity, so not all elements in $Y$ have an $X$. This means that every $x$ has a unique $y$ but not all $y$'s have an $x$, thus $|X| \leq |Y|$ *not* $|X| = |Y|$

This is a contradiction with the previously established fact

$$|\mathbb{M}| < |\mathcal{P}(\{0,1\}^*)|$$

Thus our assumption is false, and therefore not all decision problems are decidable.

---

## 5.3 An undecidable problem

To find an example of an undecidable problem, we are going to rely on the same proof technique we used earlier to show that $|\mathbb{N}| < |\mathbb{R}|$, **diagonalisation**. But first, we need to understand two key ideas for this proof to come together.

### 5.3.1 String Casting

The only data structure we are using for our Turing Machines are strings. To do useful work, we rely on representations of an object as a string to perform a computation over that object using a Turing Machine. However, we don't distinguish between different kinds of objects our strings represent.

As a result, one string can mean two completely different objects when we choose a different representation.

$$\langle 3 \rangle_{\mathbb{N},2} = 11$$
$$\langle -1 \rangle_{\mathbb{Z},2} = 11$$

This is where the idea of types come from in programming. They allow us to distinguish that even though we can have the same binary string of data, we can still have two different objects.

Most programming languages also provide a tool to convert between different types. This tool is called casting.

```
uint8_t a = 255; // In memory a is the binary string 1111 1111
int8_t b = (int8_t)a; // In memory b is the binary string 1111 1111

std::cout << a << std::endl; // prints 255
std::cout << b << std::endl; // prints -1
```

It is important to recognise that any string $w \in \{0,1\}^*$ could represent any object, and that the representation of some object $\langle o \rangle$ can be used as any other string.

### 5.3.2 Feeding TM strings into TMs

Why do are we interested in string casting? It is the tool that allows us to do incredibly strange things such as this:

Let $PAL$ be a TM that decides PALINDROME.

We can represent $PAL$ as a string, $\langle PAL \rangle$.

Let $w \in \{0,1\}^*$ be $\langle PAL \rangle$ cast as a string.

We can run the computation $C(PAL, w)$ and get the result $PAL(w)$ i.e. is the string $w$ a palindrome? But

$$w = \langle PAL \rangle$$

so

$$PAL(w) = PAL(\langle PAL \rangle)$$

We can run a Turing Machine on the string representation of itself. Even if a Turing Machine $M$ does not test for some property on Turing Machines, there is nothing stopping us from *still* running Turing Machines through $M$ and getting some result.

Using this idea, we can define the following set of languages

$$\begin{aligned}
\text{SELF-ACCEPT} &:= \{\langle M \rangle | M(\langle M \rangle) = 1\} \\
\text{NOT-SELF-ACCEPT} &:= \{\langle M \rangle | M(\langle M \rangle) \neq 1\} \\
\\
\text{SELF-REJECT} &:= \{\langle M \rangle | M(\langle M \rangle) = 0\} \\
\text{NOT-SELF-REJECT} &:= \{\langle M \rangle | M(\langle M \rangle) \neq 0\} \\
\\
\text{SELF-HALT} &:= \{\langle M \rangle | M(\langle M \rangle) \neq \infty\} \\
\text{NOT-SELF-HALT} &:= \{\langle M \rangle | M(\langle M \rangle) = \infty\}
\end{aligned}$$

> **Tip**
>
> The above languages can be a bit trippy to read the first time you see them. If we write them in english however it may become a bit clearer what exactly these languages are.
> SELF-ACCEPT is the set of all Turing Machines (as strings) that accept when the string representation of itself (cast as a normal string) is given as input.
> NOT-SELF-ACCEPT is the set of all Turing Machines (as strings) that do not accept (i.e. reject *or* loop) when the string representation of itself (cast as a normal string) is given as input.

### 5.3.3 Diagonalisation

We are going to show that NOT-SELF-ACCEPT is an undecidable problem. To do this, we are going to rely on diagonalisation again.

---

**Theorem**: The language NOT-SELF-ACCEPT is undecidable.

**PROOF SKETCH**

The set of all non-equivalent Turing Machines $\mathbb{M}$ is countable. Therefore we can enumerate it as

$$M_1, M_2, M_3, M_4, M_5, M_6, ...$$

and then we can represent this list as strings

$$\langle M_1 \rangle, \langle M_2 \rangle, \langle M_3 \rangle, \langle M_4 \rangle, \langle M_5 \rangle, \langle M_6 \rangle, ...$$

Using these two lists, we can construct the following table. The value at row $i$ and column $j$ is the result of $M_i(\langle M_j \rangle)$. We don't actually know what the result of these computations are, so for the sake of the argument we will pick essentially random values.

| | $\langle M_1 \rangle$ | $\langle M_2 \rangle$ | $\langle M_3 \rangle$ | $\langle M_4 \rangle$ | $\langle M_5 \rangle$ | $\langle M_6 \rangle$ | $\cdots$ |
|---|---|---|---|---|---|---|---|
| $M_1$ | 1 | 0 | $\infty$ | 0 | 0 | 1 | $\cdots$ |
| $M_2$ | $\infty$ | 0 | 0 | 1 | 1 | $\infty$ | $\cdots$ |
| $M_3$ | 1 | 0 | $\infty$ | $\infty$ | 1 | 1 | $\cdots$ |
| $M_4$ | 0 | 1 | $\infty$ | 0 | 0 | 1 | $\cdots$ |
| $M_5$ | $\infty$ | 1 | 1 | 1 | 1 | $\infty$ | $\cdots$ |
| $M_6$ | 0 | $\infty$ | $\infty$ | 1 | 0 | $\infty$ | $\cdots$ |
| $\vdots$ | | | | | | | $\ddots$ |

The the values on diagonal of this table are the results of running a TM on itself as input. We can therefore construct some language by taking these values and changing their results.

In this case, we will take the value and say it is 1 if $M_i(\langle M_i \rangle) \neq 1$ and 0 if it is equal to one. This constructed language is NOT-SELF-ACCEPT.

| | $\langle M_1 \rangle$ | $\langle M_2 \rangle$ | $\langle M_3 \rangle$ | $\langle M_4 \rangle$ | $\langle M_5 \rangle$ | $\langle M_6 \rangle$ | $\cdots$ |
|---|---|---|---|---|---|---|---|
| $M_1$ | 1 | 0 | $\infty$ | 0 | 0 | 1 | $\cdots$ |
| $M_2$ | $\infty$ | 0 | 0 | 1 | 1 | $\infty$ | $\cdots$ |
| $M_3$ | 1 | 0 | $\infty$ | $\infty$ | 1 | 1 | $\cdots$ |
| $M_4$ | 0 | 1 | $\infty$ | 0 | 0 | 1 | $\cdots$ |
| $M_5$ | $\infty$ | 1 | 1 | 1 | 1 | $\infty$ | $\cdots$ |
| $M_6$ | 0 | $\infty$ | $\infty$ | 1 | 0 | $\infty$ | $\cdots$ |
| $\vdots$ | | | | | | | $\ddots$ |
| NOT-SELF-ACCEPT | 0 | 1 | 1 | 1 | 0 | 1 | $\cdots$ |

If we assume that NOT-SELF-ACCEPT is decidable, then the decider $M_{NSA}$ must at some point be enumerated in the list. The row of $M_{NSA}$ is equal to the constructed NOT-SELF-ACCEPT as it is the decider for the problem.

| | $\langle M_1 \rangle$ | $\langle M_2 \rangle$ | $\langle M_3 \rangle$ | $\langle M_4 \rangle$ | $\langle M_5 \rangle$ | $\langle M_6 \rangle$ | $\cdots$ | $\langle M_{NSA} \rangle$ | $\cdots$ |
|---|---|---|---|---|---|---|---|---|---|
| $M_1$ | 1 | 0 | $\infty$ | 0 | 0 | 1 | $\cdots$ | 0 | $\cdots$ |
| $M_2$ | $\infty$ | 0 | 0 | 1 | 1 | $\infty$ | $\cdots$ | 1 | $\cdots$ |
| $M_3$ | 1 | 0 | $\infty$ | $\infty$ | 1 | 1 | $\cdots$ | 0 | $\cdots$ |
| $M_4$ | 0 | 1 | $\infty$ | 0 | 0 | 1 | $\cdots$ | $\infty$ | $\cdots$ |
| $M_5$ | $\infty$ | 1 | 1 | 1 | 1 | $\infty$ | $\cdots$ | 0 | $\cdots$ |
| $M_6$ | 0 | $\infty$ | $\infty$ | 1 | 0 | $\infty$ | $\cdots$ | $\infty$ | $\cdots$ |
| $\vdots$ | | | | | | | $\ddots$ | | |
| $M_{NSA}$ | 0 | 1 | 1 | 1 | 0 | 1 | $\cdots$ | ??? | $\cdots$ |
| $\vdots$ | | | | | | | | | $\ddots$ |
| NOT-SELF-ACCEPT | 0 | 1 | 1 | 1 | 0 | 1 | $\cdots$ | ??? | $\cdots$ |

To construct NOT-SELF-ACCEPT we need to flip the diagonal values in the table. But what appears in the table for $M_{NSA}$ on $\langle M_{NSA} \rangle$? If it is $\infty$, then NOT-SELF-ACCEPT would be 1, so in the table we need to have the value 1? If it is 1, then NOT-SELF-ACCEPT would 0, so in the table we would need to have 0?

The expected output of $M_{NSA}$ on $\langle M_{NSA} \rangle$ is unclear and contradictory.

> **Note**
>
> The above argument was not a rigourous proof. Using diagonalisation, we can show that the that behaviour of $M_{NSA}$ is ambiguous, but it doesn't necessarily imply a contradiction. We need to use a correct math proof to demonstrate that this ambiguity is in fact a mathematical problem.

---

**PROOF**

Let NOT-SELF-ACCEPT be a decision problem defined as follows

$$\text{NOT-SELF-ACCEPT}(\langle M \rangle) := \begin{cases} 1 & M(\langle M \rangle) \neq 1 \\ 0 \text{ otherwise} & \end{cases}$$

Assume, for contradiction, that there exists a decider $M_{NSA}$ for NOT-SELF-ACCEPT (abbrev. NSA)

There are two ideas we need here

1. $\text{NSA}(\langle M_{NSA} \rangle) = 1 \iff M_{NSA}(\langle M_{NSA} \rangle) \neq 1$ by the definition of NSA
2. $M_{NSA}(\langle M_{NSA} \rangle) = 1 \iff \text{NSA}(\langle M_{NSA} \rangle) = 1$ by assumption that $M_{NSA}$ is a decider for NSA

Putting these together

$$\begin{aligned} M_{NSA}(\langle M_{NSA} \rangle) = 1 &\iff \text{NSA}(\langle M_{NSA} \rangle) = 1 \iff M_{NSA}(\langle M_{NSA} \rangle) \neq 1 \\ \therefore M_{NSA}(\langle M_{NSA} \rangle) = 1 &\iff M_{NSA}(\langle M_{NSA} \rangle) \neq 1 \end{aligned}$$

We have a contradiction. Thus our assumption that there exists a decider for NOT-SELF-ACCEPT is wrong.

Therefore, there is no decider for NOT-SELF-ACCEPT

---

**Tip**

You can use any of the mentioned languages to construct a similar proof. Try it for yourself.